

Dokumentation – Lagerverwaltung

Methoden-Referenz zum POS-Projekt **Lagerverwaltung** (Java + Vaadin, Spengergasse).

Darstellung: Jede Methode zeigt ihre vollständige Signatur (Rückgabetyp + Parameter) sowie `@param`, `@return` und `@throws`, damit Vertrag und internes Verhalten erkennbar sind. Logik-Methoden stehen als ausführlicher **Javadoc-Block**; einfache Getter/Setter und Helfer sind zu kompakten Signatur-Blöcken mit `//`-Kurzkommentaren gruppiert.

Worum geht es?

Das Projekt ist eine **Lagerverwaltung** (Warenwirtschaft). Verwaltet werden Artikel in drei Kategorien: Elektronik, Lebensmittel und Kleidung. Der Aufbau folgt klaren Schichten:

- **Modell** – `Artikel` (abstrakt) mit den Unterklassen `ElektronikArtikel`,

`Lebensmittel`, `Kleidung` sowie dem Aufzählungstyp `Groesse`.

- **Verwaltung** – `LagerVerwaltung` hält die Artikelliste und bietet Anlegen/Ändern/Löschen/

Suchen, Filtern, Sortieren, Gruppieren, Auswerten, Favoriten und einen Verlauf.

- **Persistenz** – `DateiService` speichert/lädt als Text- oder CSV-Datei.
- **Anwendung** – `LagerService` (Spring-`@Service`): geteilte `LagerVerwaltung`, lädt beim

Start und speichert bei jeder Änderung.

- **GUI** – drei Vaadin-Views (`ArtikellisteView`, `StatistikView`, `VerlaufView`) im Rahmen

von `MainLayout` / `ViewTitle`, gestartet über `Application`.

- **Hilfsklassen** – `LogEintrag` (Verlaufseintrag) und `UngueltigeEingabeException`.
- **Demo & Tests** – `LagerverwaltungDemo` (Konsole), `DateiServiceTest` und

`LagerVerwaltungTest` (JUnit).

Konvention: Klassen, Methoden, Kommentare und Texte auf **Deutsch**. Ungültige Eingaben lösen die geprüfte `UngueltigeEingabeException` aus. Paket: `at.spengergasse.lager`.

Artikel (abstrakte Oberklasse)

Basis aller Artikel: Felder ID, Name, Marke, Preis, Bestand, Favorit. Konstanten `MWST_NORMAL` (0.20) und `MWST_REDUIZERT` (0.10).

```

/**
 * Erzeugt einen neuen Artikel und prüft dabei alle übergebenen Werte
 * (intern: ruft die pruefe...-Helfer auf, bevor die Felder gesetzt werden).
 *
 * @param id      eindeutige Artikelnummer (muss > 0 sein)
 * @param name     Bezeichnung (darf nicht leer sein)
 * @param marke    Hersteller/Marke (darf nicht leer sein)
 * @param preis    Nettopreis (darf nicht negativ sein)
 * @param bestand  Stückzahl im Lager (darf nicht negativ sein)
 * @throws UngeltigeEingabeException wenn einer der Werte ungültig ist
 */
public Artikel(int id, String name, String marke, double preis, int bestand)

```

```

// Getter – liefern den jeweiligen Feldwert
public int      getId()          // @return eindeutige Artikelnummer (> 0)
public String   getName()        // @return Bezeichnung
public String   getMarke()       // @return Marke / Hersteller
public double   getPreis()       // @return Nettopreis
public int      getBestand()     // @return Stückzahl im Lager
public boolean  istFavorit()     // @return true, wenn als Favorit markiert

// Setter – prüfen den Wert (pruefe...) und werfen bei Verstoß
public void setFavorit(boolean favorit) // @param favorit Favoriten-Flag (ohne Prüfung)
public void setId(int id)              // @param id neue ID (>0)    @throws
UngeltigeEingabeException
public void setName(String name)       // @param name nicht leer    @throws
UngeltigeEingabeException
public void setMarke(String marke)     // @param marke nicht leer  @throws
UngeltigeEingabeException
public void setPreis(double preis)     // @param preis ≥ 0         @throws
UngeltigeEingabeException
public void setBestand(int bestand)    // @param bestand ≥ 0      @throws
UngeltigeEingabeException

```

Hinweis zu `setId` : Ändert die ID nachträglich – kann die ID-Eindeutigkeit innerhalb einer `LagerVerwaltung` brechen (suchen/entfernen verlassen sich darauf).

```

/**
 * Berechnet den Lagerwert dieses Artikels (intern: preis * bestand).
 * @return Gesamtwert = Preis × Stückzahl
 */
public double berechneWert()

```

```

/**
 * Prüft die Verfügbarkeit (intern: bestand > 0).
 * @return true, wenn mindestens ein Stück auf Lager ist
 */
public boolean istVerfuegbar()

```

```

/**
 * Endpreis inklusive Mehrwertsteuer – von jeder Unterklasse selbst implementiert.
 * @return Bruttopreis
 */
public abstract double berechnePreis()

/**
 * @return Name der Kategorie ("Elektronik" / "Lebensmittel" / "Kleidung")
 */
public abstract String getKategorie()

```

```

/**
 * @return einzeilige Darstellung: Kategorie, ID, Name, Marke, Preis (2 Nachkommastellen),
Bestand
 */
public String toString()

// Private statische Validierungs-Helfer (genutzt von Konstruktor UND Settern)
private static void pruefeId(int id) // @throws UngueltigeEingabeException wenn id
≤ 0
private static void pruefeName(String name) // @throws UngueltigeEingabeException wenn
null/leer
private static void pruefeMarke(String marke) // @throws UngueltigeEingabeException wenn
null/leer
private static void pruefePreis(double preis) // @throws UngueltigeEingabeException wenn
negativ
private static void pruefeBestand(int bestand) // @throws UngueltigeEingabeException wenn
negativ

```

ElektronikArtikel

Elektronikgerät mit Garantiedauer und optionaler Batterie.

```

/**
 * Erzeugt ein Elektronikgerät (zunächst ohne Batterie, batterieTyp = null).
 *
 * @param garantieMonate Garantiedauer in Monaten (darf nicht negativ sein)
 * @throws UngueltigeEingabeException wenn ein Wert ungültig ist
 */
public ElektronikArtikel(int id, String name, String marke,
                        double preis, int bestand, int garantieMonate)

```

```

public int    getGarantieMonate() // @return Garantiedauer in Monaten
public void   setGarantieMonate(int garantieMonate) // @param garantieMonate ≥ 0 @throws
UngueltigeEingabeException
public String getBatterieTyp() // @return Batterietyp oder null (keine
Batterie)

```

```

/**
 * Weist dem Gerät nachträglich eine Batterie zu (intern: setzt batterieTyp).
 *
 * @param typ Bezeichnung des Batterietyps (darf nicht leer sein)
 * @throws UngueltigeEingabeException wenn der Typ leer ist
 */
public void batterieHinzufuegen(String typ)

```

```

/**
 * Entfernt die Batterie (intern: batterieTyp = null).
 */
public void batterieEntfernen()

/**
 * @return true, wenn ein Batterietyp gesetzt ist (intern: batterieTyp ≠ null)
 */
public boolean istMitBatterie()

/**
 * @return Bruttopreis (Nettopreis + 20 % MwSt)
 */
public double berechnePreis()

/** @return "Elektronik" */
public String getKategorie()

private static void pruefeGarantieMonate(int garantieMonate) // @throws
    UngueltigeEingabeException wenn negativ

```

Lebensmittel

Lebensmittel mit Haltbarkeitsdatum (**LocalDate**) und Bio-Kennzeichen.

```

/**
 * Erzeugt ein Lebensmittel.
 *
 * @param haltbarBis Haltbarkeitsdatum (darf nicht null sein)
 * @param bio true, wenn es ein Bio-Produkt ist
 * @throws UngueltigeEingabeException wenn ein Wert ungültig ist
 */
public Lebensmittel(int id, String name, String marke, double preis,
                    int bestand, LocalDate haltbarBis, boolean bio)

```

```

public LocalDate getHaltbarBis() // @return Haltbarkeitsdatum
public void      setHaltbarBis(LocalDate datum) // @param datum nicht null @throws
    UngueltigeEingabeException
public boolean   istBio() // @return true, wenn Bio-Produkt
public void      setBio(boolean bio) // @param bio Bio-Flag (ohne Prüfung)

```

```

/**
 * Berechnet, wie viele Tage zwischen Haltbarkeitsdatum und heute liegen
 * (intern: ChronoUnit.DAYS.between(haltbarBis, heute)).
 *
 * @return Tage; positiv = bereits abgelaufen, negativ = noch so viele Tage haltbar, 0 = heute
 */
public int tageAbgelaufen()

```

```

/**
 * @return true, wenn das Datum überschritten ist (intern: tageAbgelaufen() > 0)
 */
public boolean istAbgelaufen()

```

```

/**
 * Prüft, ob das Produkt demnächst abläuft (aber noch nicht abgelaufen ist).
 *
 * @param tage Vorwarnzeit in Tagen
 * @return true, wenn das Produkt innerhalb der nächsten {@code tage} Tage abläuft
 */
public boolean istBaldAbgelaufen(int tage)

```

```

/** @return Bruttopreis (Nettopreis + 10 % reduzierte MwSt) */
public double berechnePreis()

/** @return "Lebensmittel" */
public String getKategorie()

private static void pruefeHaltbarBis(LocalDate haltbarBis) // @throws
    UngeltigeEingabeException wenn null

```

Kleidung

Kleidungsstück mit Konfektionsgröße (**Groesse**) und Material.

```

/**
 * Erzeugt ein Kleidungsstück.
 *
 * @param groesse Konfektionsgröße (darf nicht null sein)
 * @param material Materialbezeichnung (darf nicht leer sein)
 * @throws UngeltigeEingabeException wenn ein Wert ungültig ist
 */
public Kleidung(int id, String name, String marke, double preis,
                int bestand, Groesse groesse, String material)

```

```

public Groesse getGroesse() // @return Konfektionsgröße
public void setGroesse(Groesse groesse) // @param groesse nicht null @throws
    UngeltigeEingabeException
public String getMaterial() // @return Material
public void setMaterial(String material) // @param material nicht leer @throws
    UngeltigeEingabeException

```

```

/**
 * Vergleicht die Größe dieses Stücks mit einer Vergleichsgröße
 * (intern: Vergleich über ordinal(), XS=0 ... XL=4).
 *
 * @param vergleich Größe, mit der verglichen wird
 * @return true, wenn dieses Stück kleiner oder gleich groß ist (XS < S < M < L < XL)
 */
public boolean istGroesseKleinerGleich(Groesse vergleich)

```

```

/**
 * Prüft auf Naturfaser (intern: Vergleich mit baumwolle/wolle/leinen/seide, klein
 geschrieben).
 * @return true, wenn das Material eine Naturfaser ist
 */
public boolean istMaterialNatuerlich()

```

```

/** @return Bruttopreis (Nettopreis + 20 % MwSt) */
public double berechnePreis()

/** @return "Kleidung" */
public String getKategorie()

private static void pruefeGroesse(Groesse groesse)    // @throws UngueltigeEingabeException
wenn null
private static void pruefeMaterial(String material)  // @throws UngueltigeEingabeException
wenn null/leer

```

Groesse (Aufzählungstyp)

Konfektionsgrößen in fester Reihenfolge: **XS, S, M, L, XL**. Die Reihenfolge (Position über `ordinal()`) wird für Größenvergleiche genutzt.

LagerVerwaltung

Hält Artikelliste und Verlauf und stellt alle fachlichen Funktionen bereit. Jede verändernde Aktion schreibt automatisch einen Verlaufseintrag.

```

/**
 * Legt eine leere Verwaltung an (intern: leere artikeLListe und leerer verlauf).
 */
public LagerVerwaltung()

/**
 * Hängt einen Verlaufseintrag an (intern von allen mutierenden Methoden genutzt).
 * @param aktion Beschreibungstext der Aktion
 */
private void log(String aktion)

```

Anlegen, Löschen, Suchen, Anzeigen

```

/**
 * Nimmt einen Artikel neu ins Lager auf (intern: Schleife prüft die ID auf Eindeutigkeit).
 *
 * @param a hinzuzufügender Artikel (darf nicht null sein)
 * @throws UngueltigeEingabeException wenn a null ist oder die ID schon vergeben ist
 */
public void hinzufuegen(Artikel a)

```

```

/**
 * Entfernt den Artikel mit der angegebenen ID (intern: lineare Suche + remove).
 *
 * @param id ID des zu löschenden Artikels
 * @return true, wenn ein Artikel gelöscht wurde; false, wenn keiner passte
 */
public boolean entfernen(int id)

```

```

/**
 * Ändert einen vorhandenen Artikel (Replace-by-ID); die ID der neuen Daten muss mit der
 * Ziel-ID übereinstimmen (intern: lineare Suche + set + Verlaufseintrag).
 *
 * @param id ID des zu ändernden Artikels
 * @param neueArtikel neue Artikeldaten (muss dieselbe ID haben)
 * @return true, wenn ein Artikel mit dieser ID gefunden und ersetzt wurde
 * @throws UngueltigeEingabeException wenn neueArtikel null ist oder die ID nicht passt
 */
public boolean aendern(int id, Artikel neueArtikel)

```

```

/**
 * Sucht den ersten Artikel mit dem angegebenen Namen (intern: equalsIgnoreCase).
 *
 * @param name gesuchter Name
 * @return der gefundene Artikel oder null, wenn keiner passt
 */
public Artikel suchen(String name)

```

```

/**
 * @return Kopie der Artikelliste (defensive Kopie – schützt die interne Liste)
 */
public ArrayList<Artikel> alleAnzeigen()

/** @return Anzahl der Artikel im Lager */
public int anzahl()

```

Filtern

```

/**
 * Liefert alle Artikel, deren Preis im Bereich liegt (Grenzen inklusive).
 *
 * @param min untere Preisgrenze
 * @param max obere Preisgrenze
 * @return Liste der passenden Artikel (ggf. leer)
 * @throws UngueltigeEingabeException wenn min größer als max ist
 */
public ArrayList<Artikel> filterNachPreis(double min, double max)

```

```

/**
 * Filtert nach Mindestbestand (intern: Schleife, bestand ≥ min).
 * @param min Mindestbestand
 * @return Liste der Artikel mit Bestand ≥ min
 */
public ArrayList<Artikel> filterNachBestand(int min)

```

```

/**
 * Filtert nach Marke (intern: equalsIgnoreCase; bei marke == null leere Liste).
 * @param marke gesuchte Marke
 * @return Liste der Artikel dieser Marke
 */
public ArrayList<Artikel> filterNachMarke(String marke)

```

```

/**
 * Mehrfach-Filter (Kategorie + Marke + Preisbereich) in einem Aufruf; leere/null Kriterien
 * werden ignoriert. Wird von der GUI-Filterleiste genutzt.
 *
 * @param kategorie Kategorienname oder null/"" für alle
 * @param marke Marke oder null/"" für alle
 * @param minPreis untere Preisgrenze
 * @param maxPreis obere Preisgrenze
 * @return Liste der Artikel, die alle gesetzten Kriterien erfüllen
 * @throws UngueltigeEingabeException wenn minPreis größer als maxPreis ist
 */
public ArrayList<Artikel> filtern(String kategorie, String marke,
                                double minPreis, double maxPreis)

```

```

/**
 * Sucht abgelaufene Lebensmittel (intern: instanceof-Prüfung + Cast + istAbgelaufen()).
 * @return Liste der bereits abgelaufenen Lebensmittel
 */
public ArrayList<Lebensmittel> abgelaufeneLebensmittel()

```

Sortieren (arbeiten auf einer Kopie – interne Reihenfolge bleibt erhalten)


```

/**
 * @return neue, nach Name aufsteigend sortierte Liste (intern: Kopie + Comparator, case-insensitive)
 */
public ArrayList<Artikel> sortiereNachName()

/**
 * @return neue, nach Preis aufsteigend sortierte Liste (intern: Double.compare)
 */
public ArrayList<Artikel> sortiereNachPreis()

/**
 * @return neue, nach Bestand absteigend sortierte Liste (intern: Integer.compare, b vor a)
 */
public ArrayList<Artikel> sortiereNachBestand()

```

Auswerten

```

/** @return Summe aller berechneWert() (Preis × Bestand) über alle Artikel */
public double gesamtWert()

/** @return Durchschnitt der Listenpreise; 0, wenn das Lager leer ist */
public double durchschnittPreis()

/** @return Summe der Stückzahlen aller Artikel */
public int gesamtBestand()

/** @return Artikel mit höchstem Preis; null, wenn das Lager leer ist */
public Artikel teuersterArtikel()

/** @return Artikel mit niedrigstem Preis; null, wenn das Lager leer ist */
public Artikel guenstigsterArtikel()

/** @return teuerstes Elektronikgerät (typgenau, kein Cast nötig); null, wenn keines vorhanden */
public ElektronikArtikel teuerstesElektronik()

/** @return teuerstes Kleidungsstück; null, wenn keines vorhanden */
public Kleidung teuersteKleidung()

```

Gruppieren & kombinierte Abfrage

```

/**
 * Gruppiert alle Artikel nach Kategorie (intern: TreeMap → Schlüssel alphabetisch sortiert).
 *
 * @return Zuordnung Kategorienname → Liste der Artikel dieser Kategorie
 */
public Map<String, ArrayList<Artikel>> gruppierenNachKategorie()

```

```

/**
 * Combo-Methode: filtert nach Mindestpreis und sortiert das Ergebnis (intern: Filter-Schleife
 * + Collections.sort nach Preis).
 *
 * @param minPreis kleinster zulässiger Preis (inklusive)
 * @return gefilterte Artikel, aufsteigend nach Preis sortiert
 */
public ArrayList<Artikel> filterUndSortierenNachPreis(double minPreis)

```

Favoriten (optionales Feature)

```

/** @return Liste aller als Favorit markierten Artikel */
public ArrayList<Artikel> filterFavoriten()

```

```

/**
 * Setzt/entfernt die Favoritenmarkierung und schreibt einen Verlaufseintrag.
 *
 * @param id ID des Artikels
 * @param favorit true = markieren, false = Markierung entfernen
 * @return true, wenn ein Artikel mit dieser ID gefunden wurde
 */
public boolean markiereFavorit(int id, boolean favorit)

```

Verlauf (optionales Feature)

```

/** @return Kopie aller Verlaufseinträge (defensive Kopie) */
public ArrayList<LogEintrag> getVerlauf()

```

Datei speichern/laden (delegiert an `DateiService`, fängt `IOException`)

```

/**
 * Lädt Artikel aus einer Textdatei und fügt sie hinzu. Defekte oder doppelte Zeilen werden
 * übersprungen und im Verlauf vermerkt.
 *
 * @param dateipfad Pfad zur Textdatei
 * @return Anzahl der erfolgreich geladenen Artikel
 * @throws UngueltigeEingabeException wenn die Datei nicht gelesen werden kann
 */
public int ladenAusDatei(String dateipfad)

```

```

/**
 * Wie {@link #ladenAusDatei}, jedoch für das CSV-Format.
 *
 * @param dateipfad Pfad zur CSV-Datei
 * @return Anzahl der erfolgreich geladenen Artikel
 * @throws UngueltigeEingabeException wenn die Datei nicht gelesen werden kann
 */
public int ladenAusCSV(String dateipfad)

```

```

/**
 * Speichert die aktuelle Artikelliste im Textformat.
 *
 * @param dateipfad Zielpfad
 * @throws UngueltigeEingabeException wenn die Datei nicht geschrieben werden kann
 */
public void speichernInDatei(String dateipfad)

/**
 * Speichert die aktuelle Artikelliste im CSV-Format.
 *
 * @param dateipfad Zielpfad
 * @throws UngueltigeEingabeException wenn die Datei nicht geschrieben werden kann
 */
public void speichernAlsCSV(String dateipfad)

```

DateiService

Speichert/lädt Artikellisten. Zwei Formate mit gleichem Feldaufbau: Text (Semikolon) und CSV (Komma, mit Kopfzeile). Trennzeichen in Feldern werden „escaped“.

```

/**
 * Schreibt eine Artikelliste im Textformat (Felder durch ';' getrennt).
 *
 * @param liste zu speichernde Artikel
 * @param dateipfad Zielpfad
 * @throws IOException bei einem Schreibfehler
 */
public static void speichern(List<Artikel> liste, String dateipfad)

```

```

/**
 * Liest eine Textdatei und erzeugt die Artikel. Inhaltlich defekte Zeilen (ungültige Zahl,
 * falscher Enum-Wert, kaputtes Datum) werden übersprungen.
 *
 * @param dateipfad Pfad zur Textdatei
 * @return Liste der eingelesenen Artikel
 * @throws IOException wenn die Datei nicht geöffnet/gelesen werden kann
 */
public static ArrayList<Artikel> laden(String dateipfad)

```

```

/**
 * Schreibt eine Artikelliste als CSV (Kopfzeile + Felder durch ',' getrennt).
 *
 * @param liste zu speichernde Artikel
 * @param dateipfad Zielpfad
 * @throws IOException bei einem Schreibfehler
 */
public static void speichernCSV(List<Artikel> liste, String dateipfad)

```

```

/**
 * Liest eine CSV-Datei (überspringt die Kopfzeile) und erzeugt die Artikel.
 *
 * @param dateipfad Pfad zur CSV-Datei
 * @return Liste der eingelesenen Artikel
 * @throws IOException wenn die Datei nicht geöffnet/gelesen werden kann
 */
public static ArrayList<Artikel> ladenCSV(String dateipfad)

```

```

/**
 * Macht Trennzeichen und Backslashes in einem Feld unschädlich (zuerst Backslash, dann
 * Trennzeichen), damit sie beim Einlesen nicht als Spaltentrenner gelten.
 *
 * @param value ursprünglicher Feldinhalt
 * @param trenner aktuelles Trennzeichen (";" oder ",")
 * @return abgesichertes Feld
 */
private static String escapeField(String value, String trenner)

/**
 * Gegenstück zu escapeField (intern: umgekehrte Reihenfolge der Ersetzungen).
 *
 * @param value escapeter Feldinhalt
 * @param trenner verwendetes Trennzeichen
 * @return ursprünglicher Feldinhalt
 */
private static String unescapeField(String value, String trenner)

```

```

/**
 * Baut aus einem Artikel eine Datenzeile, inklusive der Extra-Felder je Artikelart
 * (intern: instanceof-Verzweigung Elektronik/Lebensmittel/Kleidung).
 *
 * @param a zu serialisierender Artikel
 * @param trenner Trennzeichen zwischen den Feldern
 * @return fertige Zeile (ohne Zeilenumbruch)
 */
private static String formatZeile(Artikel a, String trenner)

```

```

/**
 * Zerlegt eine Datenzeile und erzeugt den passenden Artikel-Typ (intern: split nur an
 * nicht-escapeten Trennzeichen, dann Fallunterscheidung nach Typ-Spalte).
 *
 * @param zeile einzulesende Zeile
 * @param trenner verwendetes Trennzeichen
 * @return der rekonstruierte Artikel
 * @throws UngueltigeEingabeException bei zu wenigen Feldern oder unbekanntem Typ
 */
private static Artikel parseZeile(String zeile, String trenner)

```

LogEintrag

Ein Verlaufseintrag aus Zeitpunkt und Aktionstext.

```
/**
 * @param aktion Beschreibungstext (Zeitpunkt wird intern auf jetzt, LocalDateTime.now(),
 gesetzt)
 */
public LogEintrag(String aktion)

public LocalDateTime getZeitpunkt() // @return Zeitpunkt des Eintrags
public String        getAktion()    // @return Beschreibungstext der Aktion

/** @return "[yyyy-MM-dd HH:mm:ss] aktion" */
public String toString()
```

UngueltigeEingabeException

Eigene geprüfte Ausnahme für ungültige Eingaben (erbt von `Exception`).

```
/**
 * @param msg Fehlermeldung (wird an den Exception-Konstruktor weitergereicht)
 */
public UngueltigeEingabeException(String msg)
```

LagerService (Spring-Anbindung der GUI)

Spring-`@Service`. Hält **eine** gemeinsame `LagerVerwaltung` für alle Views, lädt beim Start aus `data/lager.csv` (oder legt Demo-Daten an) und **speichert nach jeder Änderung**. Die Geschäftslogik bleibt in `LagerVerwaltung`; der Service kümmert sich nur um die geteilte Instanz und die Persistenz.

```
/**
 * Wird von Spring nach dem Erzeugen einmal aufgerufen (@PostConstruct): lädt vorhandene
 * Daten aus der CSV-Datei oder legt beim ersten Start Demo-Daten an.
 */
void init()
```

```

// Lesende Methoden – reichen nur an die LagerVerwaltung durch (kein Speichern)
public ArrayList<Artikel> alleAnzeigen()           // @return Kopie der Artikelliste
public int anzahl()                               // @return Anzahl der Artikel
public ArrayList<Artikel> filtern(String kategorie, String marke, double minPreis, double
maxPreis)
                                                    // @return gefilterte Liste @throws
UngueltigeEingabeException wenn minPreis > maxPreis
public double gesamtWert()                         // @return Summe Preis × Bestand
public double durchschnittPreis()                 // @return Durchschnittspreis (0 wenn leer)
public int gesamtBestand()                        // @return Summe der Stückzahlen
public Artikel teuersterArtikel()               // @return teuerster Artikel oder null
public Artikel guenstigsterArtikel()             // @return günstigster Artikel oder null
public ElektronikArtikel teuerstesElektronik() // @return teuerstes Gerät oder null
public Kleidung teuersteKleidung()              // @return teuerstes Stück oder null
public Map<String, ArrayList<Artikel>> gruppierenachKategorie() // @return Gruppen je
Kategorie
public ArrayList<Lebensmittel> abgelaufeneLebensmittel() // @return abgelaufene
Lebensmittel
public ArrayList<LogEintrag> getVerlauf()         // @return Verlauf (Kopie)

```

```

/**
 * Legt einen Artikel an und speichert (intern: verwaltung.hinzufuegen + speichern).
 * @param a neuer Artikel
 * @throws UngueltigeEingabeException wenn a ungültig ist oder die ID schon existiert
 */
public void hinzufuegen(Artikel a)

/**
 * Ändert einen Artikel und speichert (intern: verwaltung.aendern(a.getId(), a) + speichern).
 * @param a geänderter Artikel (gleiche ID wie das Original)
 * @throws UngueltigeEingabeException wenn a ungültig ist
 */
public void aktualisieren(Artikel a)

/**
 * Entfernt den Artikel mit der ID und speichert bei Erfolg.
 * @param id zu löschende ID
 * @return true, wenn etwas gelöscht wurde
 */
public boolean entfernen(int id)

/**
 * Setzt/entfernt die Favoritenmarkierung und speichert bei Erfolg.
 * @param id ID des Artikels
 * @param favorit Markierung setzen (true) oder entfernen (false)
 * @return true, wenn die ID gefunden wurde
 */
public boolean markiereFavorit(int id, boolean favorit)

```

```

private void speichern() // schreibt die Liste nach data/lager.csv (legt den Ordner an);
Fehler nicht fatal
private void seedDemo() // legt sechs Beispiel-Artikel an (nur beim allerersten Start)

```

GUI (Vaadin)

Drei Views mit Navigation (`@Route` + `@Menu`), gemeinsamer Rahmen über `MainLayout` . Die Views enthalten **keine Geschäftslogik** – sie rufen den `LagerService` . Paket der Views:

`at.spengergasse.lager.ui` .

ArtikelListView (`@Route("")` – Startseite)

Tabelle aller Artikel mit sortierbaren Spalten, Suche, Mehrfach-Filter und CRUD (Neu/Bearbeiten/Löschen) sowie Favoriten-Stern.

```
/**
 * Baut die Seite auf (Werkzeugleiste, Filterleiste, Grid) und lädt die Artikel.
 * @param service eingespritzter LagerService (Datenzugriff)
 */
public ArtikelListView(LagerService service)

private HorizontalLayout erstelleWerkzeugleiste() // @return Leiste mit "Neuer Artikel" +
Suchfeld
private HorizontalLayout erstelleFilterleiste() // @return Leiste mit Kategorie/Marke/Preis-
Filter
private void erstelleTabelle() // baut die Grid-Spalten (inkl. Favorit +
Aktionen)
private void aktualisiere() // holt gefilterte Liste vom Service und
füllt das Grid
private boolean sucheTrifft(Artikel a) // @param a Zeile @return true, wenn Name
den Suchtext enthält
private void oeffneDialog(Artikel vorhanden) // @param vorhanden null = neu, sonst
bearbeiten
```

ArtikelFormDialog (Dialog zum Anlegen/Bearbeiten)

Zeigt je nach Kategorie die passenden Zusatzfelder; die Validierung kommt aus den Modell-Konstruktoren (werfen `UngueltigeEingabeException` , hier nur gefangen und angezeigt).

```
/**
 * @param vorhanden zu bearbeitender Artikel oder null für einen neuen
 * @param callback wird beim Speichern aufgerufen (hinzufügen oder aktualisieren)
 */
public ArtikelFormDialog(Artikel vorhanden, Speichern callback)

private void zeigePassendeFelder() // blendet Zusatzfelder je Kategorie
ein/aus
private void vorbefuellen() // füllt Felder beim Bearbeiten (ID +
Kategorie gesperrt)
private void aufSpeichern(Speichern callback) // baut den Artikel, ruft callback,
schließt bei Erfolg
private Artikel baueArtikel() // @return aus den Feldern erzeugter
Artikel @throws UngueltigeEingabeException bei ungültigen Eingaben
private int pflichtInt(IntegerField f, String name) // @return Pflicht-Ganzzahl @throws
UngueltigeEingabeException wenn leer
private double pflichtDouble(NumberField f, String name) // @return Pflicht-Kommazahl @throws
UngueltigeEingabeException wenn leer
```

Geschachtelte Schnittstelle `Speichern`: `void speichern(Artikel a) throws UngueltigeEingabeException` – der Rückruf, den die View beim Öffnen setzt (Anlegen bzw. Ändern + Speichern).

StatistikView (`@Route("statistik")`)

Zeigt die Auswertungen des Service (reine Anzeige).

```
/**
 * @param service eingespritzter LagerService (liefert die Kennzahlen)
 */
public StatistikView(LagerService service)

private HorizontalLayout zeile(String label, String wert) // @return eine Kennzahl-Zeile
(fetter Name + Wert)
private Span ueberschrift(String text) // @return Zwischenüberschrift
private String beschreibe(Artikel a) // @param a Artikel oder null
@return "Name (Preis)" bzw. "-"
```

VerlaufView (`@Route("verlauf")`)

Zeigt das Änderungs-Log (neueste zuerst).

```
/**
 * @param service eingespritzter LagerService (liefert getVerlauf())
 */
public VerlaufView(LagerService service)
```

Rahmen (aus dem Vaadin-Gerüst)

Application

```
/**
 * @param args Kommandozeilenargumente (an Spring Boot weitergereicht); startet die Vaadin-App
 */
public static void main(String[] args)
```

MainLayout (gemeinsamer Rahmen mit Seitennavigation)

```
/** Baut den App-Rahmen: Drawer mit Header, Navigation und Footer. */
MainLayout()

private Component createApplicationHeader() // @return Kopfbereich (Logo + App-Name)
private Component createApplicationDrawer() // @return scrollbare Seitennavigation
private Component createApplicationFooter() // @return Footer-Zeile
private SideNav createSideNav() // @return Navigation, aus den Menü-
Einträgen gebaut
private SideNavItem createSideNavItem(MenuEntry entry) // @param entry Menü-Eintrag @return
Navigationspunkt (mit/ohne Icon)
```

ViewTitle (wiederverwendbare Titelzeile)


```
/**
 * @param title anzuzeigender Titel (mit Drawer-Umschalter davor)
 */
public ViewTitle(String title)
```

Demo & Tests

LagerverwaltungDemo

```
/**
 * @param args ungenutzt - führt das Backend auf der Konsole vor (Filter, Sortierung,
 *      Auswertungen, Gruppierung, Favoriten, Datei-Roundtrip, Fehlerbehandlung)
 */
public static void main(String[] args)
```

DateiServiceTest (JUnit 5; alle Tests sind `void` ohne Parameter, nutzen `@TempDir`)

- `textFormat_roundtrip_anzahlStimmt()` — Prüft, dass nach Speichern/Laden im Textformat gleich viele Artikel vorhanden sind.
- `textFormat_roundtrip_felderStimmen()` — Prüft, dass nach dem Laden die Einzelwerte (Marke, Preis, Bestand) stimmen.
- `csvFormat_roundtrip_anzahlStimmt()` — Prüft dieselbe Anzahl-Erhaltung für das CSV-Format.
- `textFormat_semikolonImNamen_ ... ()` — Prüft, dass ein Semikolon im Namen korrekt escaped/geladen wird.
- `csvFormat_kommaImNamen_ ... ()` — Prüft, dass ein Komma im Namen/Material korrekt escaped/geladen wird.
- `textFormat_backslashImNamen_ ... ()` — Prüft, dass ein Backslash im Namen korrekt escaped/geladen wird.
- `textFormat_favoritBleibtErhalten()` — Prüft, dass die Favoritenmarkierung den Roundtrip übersteht.
- `laden_leereDate_liefertNull_Artikel()` — Prüft, dass eine leere Datei zu 0 Artikeln führt (kein Fehler).
- `laden_nichtExistenteDatei_wirftException()` — Prüft, dass eine nicht vorhandene Datei eine `UngueltigeEingabeException` auslöst.

LagerverwaltungTest (JUnit 5; testet die `aendern` -Methode)

- `aendern_existierenderArtikel_wirdErsetzt()` — Ein vorhandener Artikel wird per ID ersetzt; Preis und Anzahl stimmen danach.
- `aendern_nichtExistiert_gibtFalse()` — Ändern einer unbekannten ID liefert `false`.
- `aendern_null_wirft()` — `null` als neue Daten löst `UngueltigeEingabeException` aus.
- `aendern_idMismatch_wirft()` — Passt die ID der neuen Daten nicht zur Ziel-ID, wird geworfen.

Starten & Bauen

- Build-Datei: `app/pom.xml` (`groupId = at.spengergasse` , `artifactId = lagerverwaltung`), Java 17+ (eingestellt auf 25).
- Vaadin-App starten: `cd app` und dann `.\mvnw spring-boot:run` (erster Start 2–5 Minuten wegen Frontend-Build), dann `http://localhost:8080` .
- Laufzeit-Daten: `app/data/lager.csv` (wird beim ersten Start angelegt; steht in `.gitignore`).
- Tests ausführen: `cd app` und dann `.\mvnw test` .
- Konsolen-Demo: `LagerverwaltungDemo.main()` in der IDE starten.

Stand der Angabe-Anforderungen

Die ursprünglich offenen Punkte sind umgesetzt:

1. **GUI-Seiten** – ☒ drei Views (`ArtikellisteView` , `StatistikView` , `VerlaufView`) mit Navigation, Grid, CRUD-Formular, Suche, Mehrfach-Filter, sortierbaren Spalten und Statistik.
2. **Laden beim Start / Speichern bei Änderung** – ☒ über `LagerService` (`data/lager.csv`).
3. **JUnit-Tests für Lagerverwaltung** – ☒ `LagerverwaltungTest` (Ändern-Funktion).
4. **Ändern-Funktion** – ☒ `Lagerverwaltung.aendern( ... )` .

Mögliche nächste Schritte (optional): mehr JUnit-Tests für Filter/Sortierung/Auswertungen und Feinschliff der Formular-Validierung.